

YZV 322E · Applied Data Engineering · Spring 2026

IoT Scooter Fleet Data Pipeline

End-to-end containerized pipeline: 50 electric scooters,
real-time anomaly detection, automated batch aggregation

Yusuf Oğuz · 150220322

Yusuf Öksüzer · 150220334

Istanbul Technical University

Why does this pipeline exist?

- Scooter fleets generate thousands of telemetry events per minute
- A toppled scooter or critical battery discovered late → accident risk + revenue loss
- Manual inspection of log files is not scalable beyond a handful of vehicles
- Nightly aggregation (trip revenue, hotspot density) needs to run reliably without human trigger

Goal

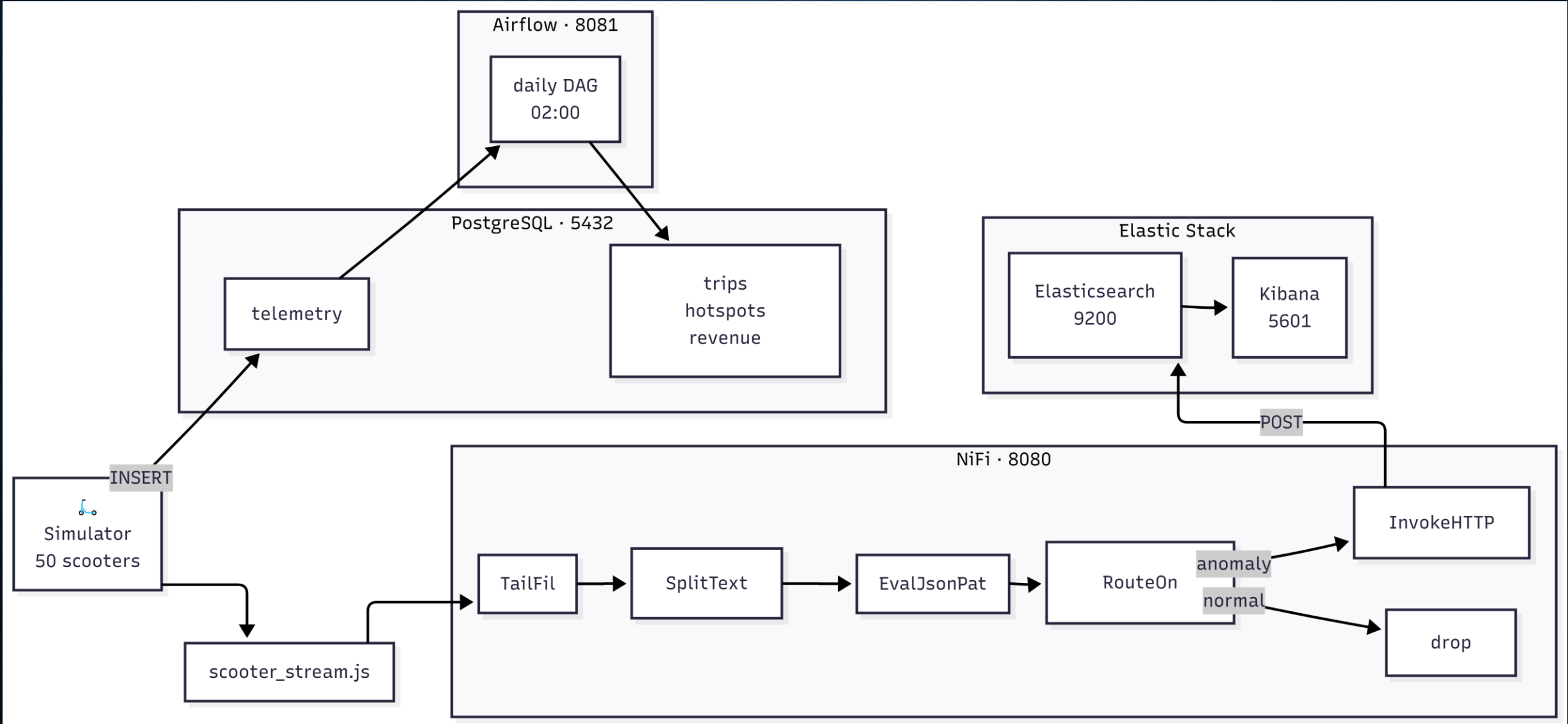
Single command startup

Real-time anomaly routing & alerting

Automated nightly aggregation

Zero host-side dependencies

System Overview



Python Simulator

50

scooters

50 rec/s

throughput

4%

anomaly rate

2

sinks

Each scooter maintains its own state:

- GPS position: random walk within Şişli-Beşiktaş bounding box
- Speed, battery %, odometer: physically realistic updates each tick
- Trip lifecycle: starts and ends trips autonomously
- Anomaly injection: topple · critical_battery · gps_loss · motor_fault

Two write paths:

PostgreSQL ← direct INSERT via psycopg2

/data/scooter_stream.jsonl → NiFi TailFile

Anomaly types:

topple

critical_battery

gps_loss

motor_fault

Apache NiFi & Real-Time Routing

1. TailFile

Reads new lines from JSONL as they arrive

2. SplitText

One FlowFile per JSON record

3. EvaluateJsonPath

Extracts anomaly_type & fault_code attributes

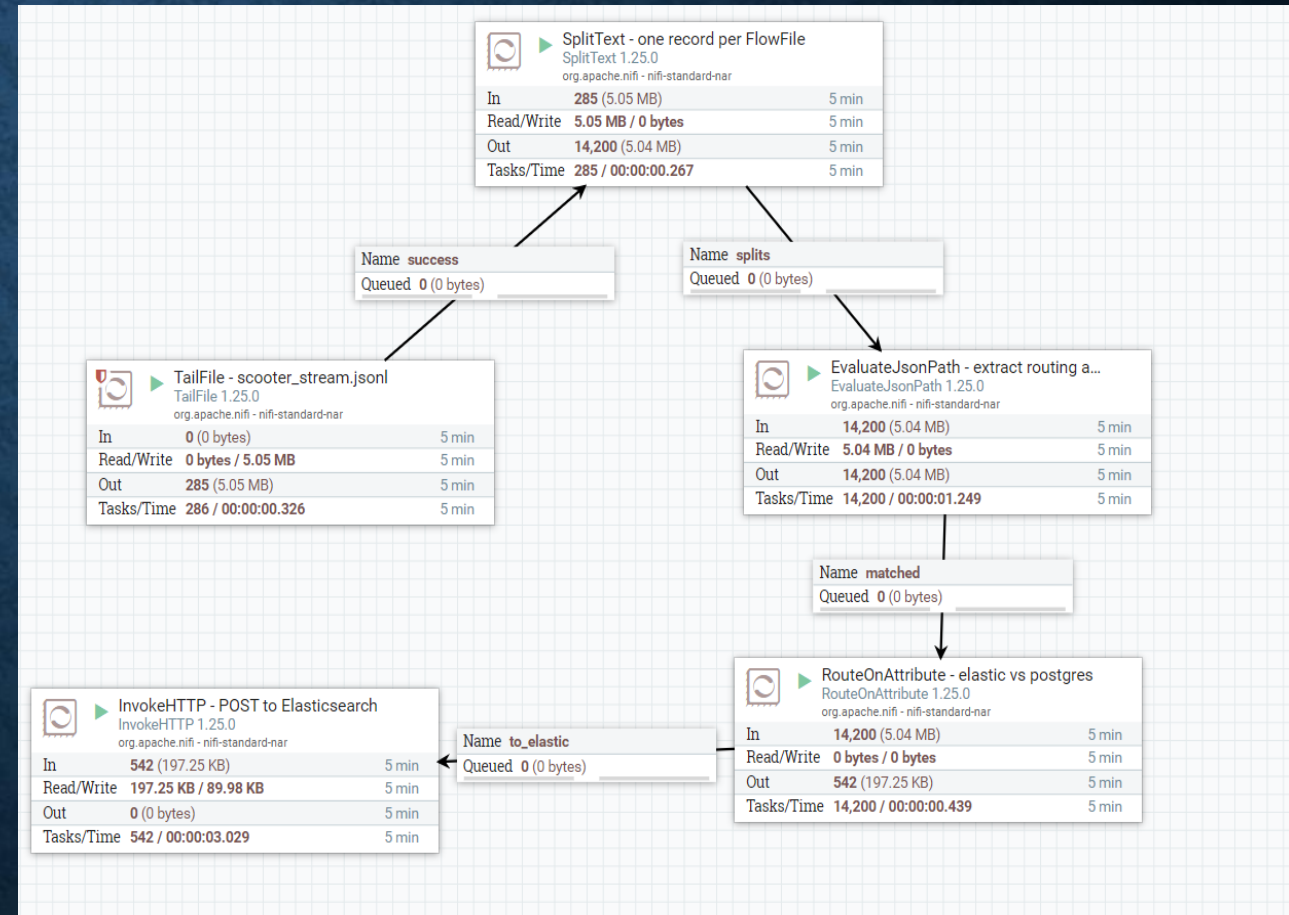
4. RouteOnAttribute

anomaly_type not empty → anomaly route

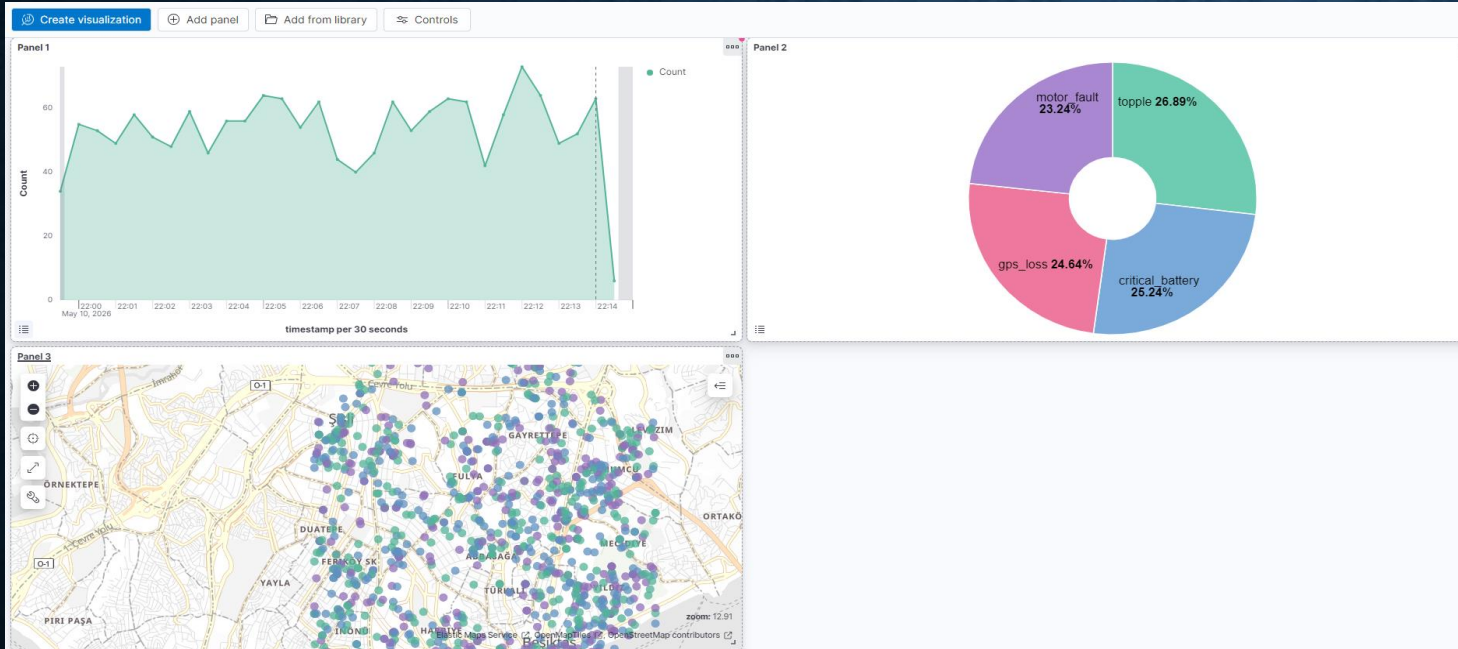
5. InvokeHTTP

HTTP POST to Elasticsearch scooter-alerts index

Normal telemetry → dropped (already in PostgreSQL via direct INSERT)



Elasticsearch & Kibana



- Index: scooter-alerts
- geo_point mapping → native map support
- Only anomaly records ingested (NiFi routes)
- 8 900+ documents after ~3 min runtime

Kibana dashboard panels:

- Area chart: anomaly event rate over time
- Donut pie: breakdown by anomaly type
- Interactive map: GPS locations plotted
- Dashboard auto-imported on first startup

PostgreSQL & Airflow

- **public.telemetry**

Raw IoT records: all scooters, all events

- **public.trips**

Trip summaries: distance, duration, revenue

- **public.daily_hotspots**

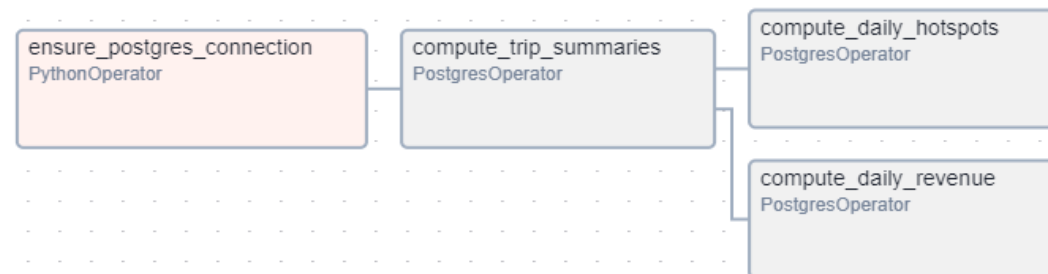
500 m grid ride-density per day

- **public.daily_revenue**

Fleet-wide revenue aggregates per day

scooter_daily_summary DAG · runs 02:00 UTC

- compute_trip_summaries
- compute_daily_hotspots
- compute_daily_revenue



	scooter_id character varying (16) 🔒	ts timestamp with time zone 🔒	speed_kmh numeric (6,2) 🔒	battery_pct numeric (5,2) 🔒	anomaly_type character varying (32) 🔒	fault_code character varying (32) 🔒
1	SC0010	2026-05-10 12:01:15.37238+00	0.00	0.85	critical_battery	[null]
2	SC0012	2026-05-10 12:01:15.372433+00	0.00	84.12	gps_loss	[null]
3	SC0015	2026-05-10 12:01:16.38558+00	0.00	64.16	gps_loss	[null]
4	SC0027	2026-05-10 12:01:16.385902+00	0.00	1.87	critical_battery	[null]
5	SC0020	2026-05-10 12:01:17.390995+00	0.00	0.97	critical_battery	[null]
6	SC0027	2026-05-10 12:01:17.391143+00	0.00	1.87	topple	[null]
7	SC0026	2026-05-10 12:01:18.396014+00	0.00	66.30	motor_fault	ERR_449
8	SC0042	2026-05-10 12:01:18.396396+00	21.64	1.42	critical_battery	[null]
9	SC0016	2026-05-10 12:01:20.404271+00	0.00	64.56	motor_fault	ERR_763
10	SC0031	2026-05-10 12:01:20.404709+00	0.00	88.19	motor_fault	ERR_764
11	SC0003	2026-05-10 12:01:21.409226+00	21.33	87.74	gps_loss	[null]
12	SC0007	2026-05-10 12:01:22.414235+00	0.00	53.95	motor_fault	ERR_290
13	SC0049	2026-05-10 12:01:22.415142+00	0.00	42.00	motor_fault	ERR_646

Why these tools?

NiFi

Low-code dataflow with a REST API that lets us automate the entire pipeline setup. (no manual UI clicks after clone.)

Airflow

DAG dependency graph cleanly models the compute_trips → hotspots/revenue fan-out, with built-in retry and scheduling.

Elasticsearch

Native geo_point support and sub-second full-text search makes it the right fit for anomaly storage and map queries.

PostgreSQL

ACID guarantees for raw telemetry; the aggregation tables are foreign-key-linked and queryable by any SQL tool.

Kibana

Tight ES integration means geo maps, time-series, and pie charts share one index pattern with zero ETL glue.

Docker Compose

Eleven services, one command. Health-check-based depends_on ensures startup order without sleep hacks.

What actually ran

194 K+

telemetry rows
in PostgreSQL

8 900+

anomaly docs
in Elasticsearch

11

containers
all healthy

1 cmd

docker compose
up --build

```
yiii4@yiii4:/mnt/d/_Development/Projects/University_Studies/yzv322(app data)/Final_project/scooter-pipeline$ docker compose start  
[+] start 12/12  
✓Container scooter-pipeline-postgres-1      Healthy      10.7s  
✓Container scooter-pipeline-elasticsearch-1 Healthy      30.8s  
✓Container scooter-pipeline-nifi-1          Started      0.3s  
✓Container scooter-pipeline-nifi-setup-1    Started      0.2s  
✓Container scooter-pipeline-simulator-1     Started      1.6s  
✓Container scooter-pipeline-airflow-init-1   Exited       16.7s  
✓Container scooter-pipeline-pgadmin-1       Started      1.7s  
✓Container scooter-pipeline-airflow-webserver-1 Started      0.4s  
✓Container scooter-pipeline-airflow-scheduler-1 Started      0.4s  
✓Container scooter-pipeline-kibana-1        Healthy      34.5s  
✓Container scooter-pipeline-es-setup-1      Started      0.5s
```

What we know is missing and what comes next

No TLS between services

All inter-container traffic is plain HTTP on the internal bridge network. Fine for dev, not for prod.

Single-threaded simulator

Scales to ~50 scooters comfortably; beyond that, async I/O or multiprocessing would be needed.

Airflow tables empty on first boot

The DAG runs at 02:00 UTC. Aggregation tables are empty until the first scheduled (or manual) trigger.

JSONL rotation edge case

NiFi TailFile tracks byte offset; log rotation can cause a brief gap until NiFi picks up the new file path.

Future directions

Replace NiFi with Apache Flink for stateful stream processing

Add Prometheus + Grafana for container health monitoring

TLS everywhere + Vault for secret management

Scale to 500+ scooters with async simulator

Use of AI Tools & Transparency Statement

What AI was used for

- Docker Compose configuration and inter-service networking
- NiFi REST API scripting and processor property debugging
- Elasticsearch index mapping (geo_point) and Kibana saved-object format
- Python simulator logic: psycopg2 batch inserts, RotatingFileHandler

What AI was NOT used for

- Architectural decisions and tool selection rationale was our own
- Report writing and all analysis and evaluation text authored by the team

All AI-generated code was reviewed, tested, and debugged by the authors. No AI output appears in this presentation or the accompanying report without human verification. Tool used: Claude (Anthropic).

Thank you

Repository

github.com/yusuf-oguz/yzv322_final_project

Services on localhost

NiFi :8080 · Kibana :5601 · Airflow :8081 · pgAdmin :5050 · ES :9200

Questions?